

真实模型试算与共享方法回查

两个完整实例与一个不等宽 head 定点检查

2026 年 9 月 24 日

结论：本轮 22 个汇总工况支持保持 Step 1 提取与 Step 2 方法。Ministral 3 8B (2512) 和 Qwen3.6-35B-A3B 各完成 10 个工况；MiMo-V2.5-Pro 仅完成 $L = 1024$ 的两个 Attention 工况。以本地 HEAD / Step 2 审阅提交 2eb7e7c 为起点，无权重下载、模型运行或性能测量。

1 先看实际结果：比例相同也有信息

以下均为精确 RI。FFN 顺序为 $B = (8, 64, 512)$ ；Attention 顺序为 $L = (1024, 16384, 131072)$ ，即 (1K, 16K, 128K)， $K = 1024$ 。QKV 为一个输入 token，权重在窗口前驻留。

主参考：局部接收端累计 ports

Workload	Ministral 3 8B (2512)	Qwen3.6-35B-A3B
QKV Projection	∞	∞
FFN / MoE	$\frac{1}{16}, \frac{1}{2}, 4$	$\frac{1}{16}, \frac{1}{2}, 4$
Attention Prefill	$\frac{2177}{128}, \frac{32897}{128}, \frac{262273}{128}$	$\frac{2177}{64}, \frac{32897}{64}, \frac{262273}{64}$
Attention Decode	32, 512, 4096	64, 1024, 8192

对照：整矩阵阶段入口 operator

Workload	Ministral 3 8B (2512)	Qwen3.6-35B-A3B
QKV Projection	∞	∞
FFN / MoE	$\frac{3}{3584}, \frac{3}{448}, \frac{3}{56}$	$\frac{5}{768}, \frac{5}{96}, \frac{5}{12}$
Attention Prefill	$\frac{1281}{128}, \frac{16641}{128}, \frac{131329}{128}$	$\frac{1537}{128}, \frac{16897}{128}, \frac{131585}{128}$
Attention Decode	18, 258, 2050	20, 260, 2052

两款 FFN 的局部 RI 均为 $B/128$ ，但 Dense 三矩阵容量为 168 MiB，单专家为 3 MiB，相差 56 倍。同一 L 下，两款 Attention 的 ports 输入与调用数完全相同，Qwen 的 KV 写入只有一半，因而局部 RI 为两倍。这些抵消来自真实维度与所声明映射，不是遗漏模型差异。

阅读与数据边界 本稿给出四类代表工况的中文代入及 MiMo 尾块检查；全部分项的两套 (Q_S, Q_R, RI) 、调用、容量与写入片保存在 `data/results.json` 和 `data/results.csv`。Byte 与分数为原始精确量，显示单位单独转换，1 KiB = 1024 Byte、1 MiB = 2^{20} Byte。各行窗口不同，不能直接相加作为端到端需求。

2 原始结构回查与计算接口

固定模型、层与实现 层号从 0 起。模型 revision 如下，未换 checkpoint:

Ministral 3 8B Base 2512, 第 0 层 full GQA + Dense:
d4883f9b36aa2e5d775730d3fdb3d30de51a8ef
Qwen3.6-35B-A3B, 第 3 层 full GQA + MoE:
995ad96eacd98c81ed38be0c5b274b04031597b0
MiMo-V2.5-Pro, 第 7 层 global GQA:
21d1ecfec7bd70f31be25ca49d7edd21f003659

前两者的 Transformers 静态实现固定于 2c4914fb939fe9de0d8e7a798af4684d552f18b4; MiMo remote code 与 checkpoint 同 revision。共享方法沿用 WS128-INT8-semantic-banks-v1, 未修改版本。

模型	D	H_q	H_{kv}	d_{QK}	d_V	F
Ministral	4096	32	8	128	128	14336
Qwen3.6	2048	16	2	256	256	512
MiMo (仅 Attention)	6144	128	8	192	128	不求值

不是只相信 models.json Ministral 的 HF text_config 与原生 params.json 逐键一致; 原生 v_head_dim=null 在固定实现中对应 V 采用 128, 未按零宽处理。Qwen 配置第 3 层是 full_attention, 原生 q_proj 输出 $16 \times 256 \times 2 = 8192$, 按每头切开 query 与 gate; 专家张量为 [256, 1024, 2048] 的 gate/up 和 [256, 2048, 512] 的 down。主对象只取其中一个专家, 不乘 8 个路由命中或 256 个专家。MiMo 的 hybrid_layer_pattern[7]=0, 全局层没有 sink; 192 是完整 QK 维度, 不能以 64 维部分 RoPE 或 $D/H_q = 48$ 替代。证据定位见文末及 data/checks.json。

角色与交接 K/V 在 GQA 扩展前写入, 每 KV head 一份: Ministral 保存 RoPE 后 K; Qwen 保存归一化、RoPE 后 K; MiMo 保存部分 RoPE 后的完整 192 维 K 与已乘 0.612 的 V。每个 head 的 K 排为 $i \times d_{QK}$ (token 沿输出行), V 排为 $d_V \times i$ (token 沿输入列)。软件 cache 布局不直接规定 CIM 的矩阵方向。

各角色参考均为 INT8 / 1 Byte; 原生 BF16/FP8 记录未覆盖, 也不作精度验证。语义矩阵独立分块, 源代码融合仅需切片或重排, 不合并接收端、不复制权重。部分和、norm/RoPE、softmax、SiLU 与逐元素门控在数字接口完成; 只有成为下一次矩阵输入的 down 向量或 Attention 系数再计入 Q_S 。所有计算假定状态有足够驻留空间, 未产生容量重载。

3 QKV: 真实矩形、额外 gate 与静态端点

Minstral: 原生结构到分块 输入为 $x \in \mathbb{Z}^{4096}$, resident 权重分别为 $W_Q: 4096 \times 4096$, $W_K, W_V: 1024 \times 4096$ 。每个输入维分为 32 个 128 维片; 输出块数为 32, 8, 8, 所有尾块均完整。一个 token 使每个 tile 求值一次。

分项	tile 数/调用数	Q_S^{ports} (Byte)	独立入口	Q_S^{op} (Byte)	Q_R (Byte)
Q	1024	131072		4096	0
K	256	32768		4096	0
V	256	32768		4096	0

于是两套汇总分别为

$$(Q_S^{\text{ports}}, Q_R^{\text{ports}}, \text{RI}^{\text{ports}}) = [4096(32 + 8 + 8), 0, \infty] = (196608, 0, \infty),$$

$$(Q_S^{\text{op}}, Q_R^{\text{op}}, \text{RI}^{\text{op}}) = (3 \times 4096 - 2 \times 4096, 0, \infty) = (4096, 0, \infty).$$

三分项 operator 输入是同一 x , 汇总扣除 8192 Byte 重叠; ports 的不同接收端直接相加。权重有效容量与 tile 分配容量均为 $1536 \times 16384 = 25165824$ Byte (24 MiB), 但本窗口写入为零。

Qwen: 完整保留 output gate 语义矩阵 Q、G、K、V 分别为 $4096 \times 2048, 4096 \times 2048, 512 \times 2048, 512 \times 2048$ 。Q/G 在原生 q-projection 中按 head 打包; 独立检查验证重排后全部行恰好覆盖一次。输入分 16 块, 输出块数为 32, 32, 4, 4:

分项	tile 数/调用数	Q_S^{ports} (Byte)	独立入口	Q_S^{op} (Byte)	Q_R (Byte)
Q	512	65536		2048	0
G (output gate)	512	65536		2048	0
K	64	8192		2048	0
V	64	8192		2048	0

ports 汇总为 $(147456, 0, \infty)$; operator 从 4×2048 扣除 3×2048 , 为 $(2048, 0, \infty)$; 共 1152 次调用, 权重容量 18 MiB。若错误删去 G, 会少 65536 Byte 局部输入、512 次调用与 8 MiB resident 权重, 静态 RI 却仍是 ∞ 。因此只检查 RI 无法发现这一结构遗漏。

为什么需求变而比例不变 两款 QKV 的输入、权重规模、调用数都不同; $Q_R = 0$ 仅来自权重预驻留的窗口。KV 建立属于 Attention 行, 不是这里的权重更新; G 是后续输出门控, 不是 query 或 KV head。本行不含输出投影 W_O , 也不拆成 Prefill/Decode 两行。

4 FFN / 单路由专家：一次装载、服务 B 个向量

Ministral, $B = 8$ 的完整代入 原生 SwiGLU 的 W_g, W_u 为 14336×4096 , W_d 为 4096×14336 。前两者各 112×32 个 tile, down 为 32×112 。三者权重在本窗口各装载一次, 随后留驻服务 8 个向量。 $z = \text{SiLU}(W_g x) \odot (W_u x)$ 重定标后是 down 的 streaming 输入。

矩阵	Q_S^{ports} (Byte)	Q_S^{op} (Byte)	Q_R (Byte)	调用数
gate	$8 \times 112 \times 4096 = 3670016$	32768	58720256	28672
up	3670016	32768	58720256	28672
down	$8 \times 32 \times 14336 = 3670016$	114688	58720256	28672

$$(Q_S^{\text{ports}}, Q_R^{\text{ports}}, \text{RI}^{\text{ports}}) = (11010048, 176160768, 1/16),$$

$$(Q_S^{\text{op}}, Q_R^{\text{op}}, \text{RI}^{\text{op}}) = (147456, 176160768, 3/3584).$$

operator 汇总 $8(4096 + 14336) = 147456$ Byte; gate/up 共享 x , 扣除一份 8×4096 , 但 down 的 114688 Byte 不能再去掉。共 10752 个 resident tile、86016 次求值; 有效/分配容量均为 168 MiB。

Qwen, 单专家 $B = 8$ 打包 gate/up 切开为两个 512×2048 , down 为 2048×512 。各矩阵 64 个 tile、1048576 Byte 权重, 8 向量各产生 65536 Byte 局部输入和 512 次调用。总计

$$(Q_S^{\text{ports}}, Q_R^{\text{ports}}, \text{RI}^{\text{ports}}) = (196608, 3145728, 1/16),$$

$$(Q_S^{\text{op}}, Q_R^{\text{op}}, \text{RI}^{\text{op}}) = (8(2048 + 512), 3145728, 5/768).$$

operator 分项为 (16384, 16384, 4096) Byte, 扣除共享输入 16384 Byte。192 个 resident tile 占 3 MiB, 求值 1536 次。路由器、共享专家及其输出门不属于这个单专家对象。

三档 B: 变化与不变 局部输入与调用随 B 成比例; 一次权重装载、最终容量、tile 数均不变。两模型 D, F 都整除 128, 因此每个矩阵均有 $Q_S^{\text{ports}} = BDF/128$ 、 $Q_R = DF$, 汇总也为

$$\text{RI}^{\text{ports}} = \frac{3BDF/128}{3DF} = \frac{B}{128} = \left(\frac{1}{16}, \frac{1}{2}, 4 \right).$$

Dense 与单专家的 ports 输入、写入、tile 数和调用数均相差 56 倍, 这个因子在比值中抵消。operator 则为 $B(D+F)/(3DF)$, 没有相同抵消。这里的相同局部 RI 依赖整除、三矩阵独立分块、等角色 Byte 和同一驻留期; 不推广为所有 FFN 的无条件恒等式。

5 Attention Prefill: 从空状态建立因果前缀

Ministral, $L = 1024$ 的完整代入 8 个 KV head 各持一份 K 和 V, 每份服务 4 个 query heads。前缀 i 的 resident K 为 $i \times 128$, V 为 $128 \times i$ 。K 分为 $\lceil i/128 \rceil \times 1$ 个 tile; V 为 $1 \times \lceil i/128 \rceil$ 。每个 token 先追加 K 的 1×128 行和 V 的 128×1 列, 再求值全部 32 个头。

$$S_{128}(1024) = 128(1 + 2 + \cdots + 8) = 4608, \quad P(1024) = \sum_{i=1}^{1024} i = 524800.$$

QK 的 query 在每个序列输出块重复接收, AV 的系数长度为 i :

$$\begin{aligned} Q_{S,QK}^{\text{ports}} &= 32 \times 128 \times 4608 = 18874368, \\ Q_{S,QK}^{\text{op}} &= 32 \times 128 \times 1024 = 4194304, \\ Q_{S,AV}^{\text{ports}} &= Q_{S,AV}^{\text{op}} = 32 \times 524800 = 16793600, \\ Q_{R,K} &= Q_{R,V} = 1024 \times 8 \times 128 = 1048576. \end{aligned}$$

因此, Byte 三元组和 RI 为

$$\begin{aligned} (Q_S^{\text{ports}}, Q_R^{\text{ports}}, \text{RI}^{\text{ports}}) &= (35667968, 2097152, 2177/128), \\ (Q_S^{\text{op}}, Q_R^{\text{op}}, \text{RI}^{\text{op}}) &= (20987904, 2097152, 1281/128). \end{aligned}$$

QK、AV 各调用 $32 \times 4608 = 147456$ 次, 总计 294912。最终各有 64 个 K/V tile, 总容量 2 MiB; 本窗口从空开始且只追加一次, 所以累计写入刚好等于最终有效容量。这不是把容量一般定义成 Q_R 。

有效尾块不能被最终整除掩盖 虽然最终 $L = 1024$ 整除 128, 前 127 个前缀以及每个块的增长期仍有尾块。例如 $i = 129$ 时 K 的输出块为 $(128, 1)$, 每块仍接收完整 128 维 query; V 的输入块为 $(128, 1)$, 第二次调用只接收 1 个有效系数。这解释了 $S_{128}(L)$ 与 $P(L)$ 是不同累计量, 不能用同一个近似代替。

Qwen 与 Ministral 为什么输入相同 Qwen 的 $H_q d_{QK} = 16 \times 256 = 4096$, 与 Ministral 相同; $H_q \lceil d_V/128 \rceil = 16 \times 2 = 32$ 也相同。故两者对任意同一 L 的 ports QK/AV 输入逐项相等。调用因子也相同: $H_q \lceil d_{QK}/128 \rceil = 32$ 和 $H_q \lceil d_V/128 \rceil = 32$ 。但 Qwen 每 token 只写 $2(256 + 256) = 1024$ Byte, Ministral 为 2048 Byte。在 1K 下, Qwen 的三元组为 ports $(35667968, 1048576, 2177/64)$, operator $(12591104, 1048576, 1537/128)$ 。

大长度计算 一般令 $L = 128m + r$, 用 $S_{128}(L) = 128m(m+1)/2 + (m+1)r$ 和 $P(L) = L(L+1)/2$ 。128K 两模型的 ports 输入均为 550026346496 Byte, 调用数 4299161600; Ministral/Qwen 的 KV 写入分别 256/128 MiB。脚本使用整数与紧凑求和, 不创建 $L \times L$ 矩阵。

6 Attention Decode: 只收费本次追加, 并比较窗口

Ministral, 可见 $L = 1024$ 初始已有 1023 个 token; 本次追加 K 的 8 份 1×128 和 V 的 8 份 128×1 , 分别写 1024 Byte。最终每头 K 为 1024×128 、V 为 128×1024 , 各 8 个 tile。32 个 query heads 的一次求值为

分项	Q_S^{ports} (Byte)	Q_S^{op} (Byte)	本次 Q_R (Byte)	调用数
QK	$32 \times 8 \times 128 = 32768$	4096	1024	256
AV	$32 \times 1024 = 32768$	32768	1024	256

$$(Q_S^{\text{ports}}, Q_R^{\text{ports}}, \text{RI}^{\text{ports}}) = (65536, 2048, 32),$$

$$(Q_S^{\text{op}}, Q_R^{\text{op}}, \text{RI}^{\text{op}}) = (36864, 2048, 18).$$

初始有效容量为 2095104 Byte, 最终为 2097152 Byte, 分配容量也为 2 MiB; $Q_R = 2048$ Byte 不是整个历史 cache 的重写。

Qwen 的不同方向与共享 K 为 1024×256 , 每 KV head 8×2 个 tile; V 为 256×1024 , 每头 2×8 。16 个 query heads 仍在 QK/AV 各调用 256 次、各接收 32768 Byte。追加 K 为两份 1×256 (各切两个 1×128), V 为两份 256×1 (各切两个 128×1)。总写入 1024 Byte、最终容量 1 MiB: ports 三元组 (65536, 1024, 64); operator (20480, 1024, 20)。

同一 L, Prefill 与 Decode 的定量关系 Prefill 写入是 Decode 的 L 倍; Prefill 输入是从 1 到 L 每次 Decode 输入的和, 因此 Prefill RI 是这些单步 RI 的均值 (这里每步追加字节相等), 而不是最后一步的 RI。对两款主模型的固定 L (均为 128 的倍数), $d_{QK} = d_V = d$ 且 d 整除 128, 令 $g = H_q/H_{kv}$:

$$\text{RI}_{\text{pre}}^{\text{ports}} = g \frac{2L + 129}{512}, \quad \text{RI}_{\text{dec}}^{\text{ports}} = g \frac{L}{128}, \quad \frac{Q_{S,\text{pre}}^{\text{ports}}}{Q_{S,\text{dec}}^{\text{ports}}} = \frac{2L + 129}{4},$$

$$\text{RI}_{\text{pre}}^{\text{op}} = \frac{g}{2} + \frac{g(L + 1)}{4d}, \quad \text{RI}_{\text{dec}}^{\text{op}} = \frac{g}{2} + \frac{gL}{2d}.$$

1K 的 ports 比值为 $\text{RI}_{\text{pre}}/\text{RI}_{\text{dec}} = 2177/4096$; 输入累计比为 $2177/4 = 544.25$, 写入比为 1024。故 Decode RI 约为 Prefill 的两倍, 不能说它们总需求或执行时间相近。

差量检查的适用范围 逐项验证 $\mathcal{D}_{\text{pre}}(L) - \mathcal{D}_{\text{pre}}(L - 1) = \mathcal{D}_{\text{dec}}(L)$, $\mathcal{D}$ 为两种 Q_S 、 Q_R 或调用数, **不对 RI 相减**。这一关系仅对应本参考的逐前缀调用与同一 append 组织; 换成批量求值、重新分块、重载或复制状态时须重新计数, 不能把差量恒等式当成所有实现的性能规律。

7 MiMo 定点: $192=128+64$, 保留有效字节和调用

原生 global GQA 与布局 只检查第 7 层、 $L = 1024$ 。 $H_q/H_{kv} = 128/8 = 16$, $d_{QK} = 192, d_V = 128$; 原生融合 QKV 宽度为 $24576 + 1536 + 1024 = 27136$ 。投影仅作结构核对, 本轮不计算 MiMo QKV 或 FFN。每 KV head 的 K 为 1024×192 , 分成 8×2 个 tile, 输入宽度为 $(128, 64)$; V 为 128×1024 , 有 8 个 tile。每次追加 K 切成 1×128 和 1×64 , V 为 128×1 。8 个头合计 24 个逻辑写入片, 不代表 24 次物理写事务。

两个窗口的分项原始 Byte

窗口	分项	Q_S^{ports}	Q_S^{op}	Q_R	调用数
Prefill	QK	113246208	25165824	1572864	1179648
	AV	67174400	67174400	1048576	589824
Decode	QK	196608	24576	1536	2048
	AV	131072	131072	1024	1024

Prefill 的 QK 输入为 $128 \times (128 + 64) \times 4608$, 调用为 $128 \times 2 \times 4608$; AV 输入为 128×524800 , 调用为 128×4608 。Decode 的 QK 输入为 $128 \times 8 \times (128 + 64)$, 调用为 $128 \times 8 \times 2$; AV 输入为 128×1024 , 调用为 128×8 。两种边界的汇总必须分别展示:

Prefill ports: (180420608, 2621440, 2753/40),

Prefill operator: (92340224, 2621440, 1409/40),

Decode ports: (327680, 2560, 128),

Decode operator: (155648, 2560, 304/5).

尾片没有消失, 也没有补成有效 128 Byte Decode QK 的 2048 次调用中, 1024 次接收 128 Byte, 1024 次接收 64 Byte; Prefill QK 的两类调用各为 589824 次。若误将尾片补满, Decode 会多计 $1024 \times 64 = 65536$ Byte; 若错误省掉尾片调用, 则会漏掉 1024 次服务。64 Byte 只是有效逻辑输入长度, 不足以推出完整硬件服务时间减半; Task I 吞吐保持原样。

最终有效 KV 为 $8 \times 1024 \times (192 + 128) = 2621440$ Byte (2.5 MiB); K 128 个 tile、V 64 个 tile, 共分配 3145728 Byte (3 MiB)。Prefill 累计写入 2.5 MiB, Decode 仅追加 2560 Byte; 三个量不混用。

维度与分块分别影响什么 H_q 放大 query/系数服务, H_{kv} 决定唯一 KV 写入与容量。 d_{QK} 决定 query 字节、K 宽度和输入分块; d_V 决定 V 写入与 AV 的输出块重放。MiMo 在固定整块 L 下, QK 和 AV 的局部 Decode RI 均为 128, 但两项需求不同且调用数为 2:1。此处比例相同也不能抹去半宽输入片的资源影响。

8 方法回查：哪些结论可继续使用

独立检查怎样避免自证 主计算使用共享 `counting.py`；独立检查从原始配置读取维度，以三条路径建立预期值：

1. 对真实矩阵直接切行列区间，累加每块有效输入、resident 元素和调用；以“源阶段、向量编号”去重 operator 的同源输入，保留 down 新阶段。
2. Attention 将循环反过来：一个绝对 token tile 从出现到 L 的存活期中，每种部分宽度出现一次，长满后在剩余前缀重复调用。按存活期累加 QK/AV tile，不使用主程序的三角数或 ceiling 闭式生成预期值。
3. 三个模型的真实 head 维度下，在 $L = 1, 127, 128, 129$ 显式枚举 query-head / KV-head / tile / 输入坐标与追加坐标，确认共享状态和尾片；再检查前缀差量。

22 个主工况的两边界与所有分项一致；12 个边界点一致；7 个主长度 Decode 的 Prefill 差量一致。所有大长度仅使用标量循环或紧凑直方图，不输出逐元素 Attention 事件文件。原始文件哈希、实现行号、独立结果及有效输入宽度对应调用数均保存在 `data/checks.json`。

一个足够小的映射对照 Qwen 1K Decode 主参考只保留 2 份 KV。若改成每个 query head 都持久保存一份 KV，共有 16 份，输入与调用不变，但每步写入与容量均乘 $g = 8$ ：

映射	ports Q_S (Byte)	ports Q_R (Byte)	ports RI	有效 KV 容量
一份 KV / KV head (主参考)	65536	1024	64	1 MiB
一份 KV / query head (对照)	65536	8192	8	8 MiB

operator 同样从 (20480, 1024, 20) 变为 (20480, 8192, 5/2)。这不是原计数错误；它换了状态复制方式，说明 GQA 共享带来的 RI 优势依赖该映射。对照未替代主参考，也不以潜在并行性推断延迟收益。

三类判断 **明确计数错误：本轮未发现**。固定配置支持 gate、专家宽度、原生 head 和层型，独立切片支持共享公式、追加次数与尾块处理。**合理参考映射的结果**：FFN 的 $B/128$ 、两主模型 Attention 同输入、逐前缀 RI 关系，均可作为后续统一需求计算的基线。**换执行方式才改变的项**：状态复制、容量重载、跨矩阵尾块合并、不同 Prefill 调度或实际写入粒度，必须声明后再求需求/服务代价。

能支撑与尚不能支撑的后续结论 “足够空间、独立矩阵、逐前缀”可描述本轮全部所选矩阵任务，未出现结构不可表达的情形。它不证明所有硬件有足够容量，或都能有效执行 K 行 / V 列小片更新。结果已按组件保留，可继续检查聚合资源、更新粒度与尾块服务；总 RI 不直接换算为单 macro 延迟、器件排名或性能分区。本轮无需修改 Step 1/2、Table II(a) 或 Task I。Step 3 审阅通过后具备进入 Step 4 的方法与数据条件；本轮到此停止。

9 证据定位与复算入口

本地原始证据 下列路径相对任务根目录；原件只读，未执行模型实现。固定 revision、SHA-256 与抽查结果见 `table_I Ib/pilot/data/config.json`、`data/checks.json`（后者相对 `pilot`）。

1. Ministral: `literature/02_ministral3_8b/raw/config.json` 的 `text_config`，及同目录 `params.json`；`literature/00_shared/raw/transformers/modeling_ministral3.py`: 111–173 行 Attention 构造、RoPE 与 cache update；176–188 行三矩阵 MLP；90–101 行 GQA 扩展与两次矩阵乘。
2. Qwen: `literature/03_qwen36_35b_a3b/raw/config.json` 的 `head`、`layer_types`、`attn_output_gate`、专家键；`literature/00_shared/raw/transformers/modeling_qwen3_5_moe.py`: 751–825 行 q-projection 双宽输出、逐头切分、norm/RoPE、cache、sigmoid gate；845–882 行单专家 gate/up 打包与 down；904–923 行独立共享专家分支；947 行起选中层型。
3. MiMo: `literature/06_mimo_v25_pro/raw/config.json` 的全局层型、192/128 head、`attention_value_scale` 和 sink 配置；同目录 `modeling_mimo_v2.py`: 215–288 行维度、融合 QKV 与 sink 条件，304–317 行 Value 缩放、部分 RoPE 与 cache update，389–406 行 global/SWA 选择。

产物与生成关系

pilot 内路径	内容
<code>data/config.json</code>	模型版本、层、角色精度、共享方法、输入快照与哈希
<code>data/results.json</code> / <code>results.csv</code>	22 个工况；每分项两套需求、调用、容量、分块/追加
<code>data/checks.json</code>	原始结构审计、独立结果、边界事件和调用宽度分布
<code>PREVIEW.md</code>	两款完整实例的四行结果、固定 B/L 顺序
<code>scripts/generate.py</code>	共享主计算、精确数据和两张预览表的生成/一致性检查
<code>scripts/check.py</code>	原始配置、真实 tile、存活期和小规模事件的独立检查
<code>scripts/build.sh</code> / <code>render.py</code>	复算、独立 PDF 编译、页面渲染
<code>data/pdf_qa.json</code>	最终 PDF 哈希、页数和逐页目视核验记录

上表 `results.csv` 实际位于 `data/`，`render.py` 位于 `scripts/`。`tex/pilot.zh.tex` 是独立中文入口；`output/pilot.zh.pdf` 是最终研究稿。分项 operator 是独立入口，聚合去重另存；不能直接相加得到 QKV/FFN 总量。

从仓库根复算与编译

```
TASK2_PYTHON=/opt/anaconda3/bin/python
sh tasks/task2_table_II_workloads/table_I Ib/pilot/scripts/build.sh
```

执行时将环境变量与 `sh` 放同一命令行，或先导出该变量。默认检查不覆盖 JSON/CSV/表格，显式 `--emit` 才刷新派生文件。共享 28 项合成检查、Table II(a) 数据/表格一致性与 Step 1 原始来源检查也纳入构建。渲染使用 `scripts/render.py` (`pypdfium2`)，逐页图保存在局部忽略的 `tmp/pdfs/`。

状态与保留 `STEP3_REVIEW.zh.md` 位于任务根目录；README 和研究配置记录“Step 2 已通过；Step 3 完成待审阅”。Step 1/2 的旧审阅原稿、共享文件中的当时状态作为历史保留，不为状态更新重编历史 PDF。没有 Git 提交、推送或 Step 4 分工。